HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO CUỐI KỲ

Đề tài: Hệ thống phát hiện rò rỉ khí gas, báo cháy

Nhóm học phần 06

Nhóm bài tập lớn 10

Các thành viên:

Đỗ Đức Cảnh - B22DCCN086

Đỗ Văn An - B22DCCN002

Trần Quang Huy - B22DCCN397

Trần Quang Huy - B22DCCN398

Hà Nội, 11/2025

Mục lục

I. GIỚI THIỆU ĐỀ TÀI 1
1.1. Bối cảnh và tính cấp thiết 1
1.2. Giới thiệu về hệ thống phát hiện rò rỉ khí gas 1
1.3. Cấu trúc tổng quan 2
II. CÔNG NGHỆ SỬ DỤNG 4
2.1 Bảng tổng hợp các thành phần công nghệ chính 4
2.2. Vai trò chi tiết của MQTT trong hệ thống 5
2.3. Vai trò chi tiết của WebSocket trong hệ thống 5
III. PHÂN TÍCH YÊU CẦU - TÍNH NĂNG 7
3.1. Yêu cầu chức năng 7
3.2. Yêu cầu kỹ thuật 7
3.3. Các tính năng triển khai 8
3.4. Các yêu cầu phi chức năng 9
IV. PHÂN TÍCH THIẾT KẾ 11
4.1. Kiến trúc hệ thống 11
4.2. Thiết kế giao tiếp phần cứng 13
4.3. Thiết kế web điều khiển 14
4.4. Nguyên lý hoạt động 26
V. KẾT QUẢ ĐẠT ĐƯỢC 29
5.1. Hoạt động hệ thống 29
5.2. Cập nhật firmware từ xa 34
VI. KẾT LUẬN – HƯỚNG MỞ RỘNG 37
6.1. Kết luận 37
6.2. Hướng mở rộng 37

Phân công nhiệm vụ nhóm

Mã sinh viên	Họ tên	Nhiệm vụ
B22DCCN002	Đỗ Văn An	Tìm hiểu yêu cầu kỹ thuật, công nghệ sử dụng, thiết kế cấu trúc tổng quan hệ thống.
B22DCCN086	Đỗ Đức Cảnh	Lập trình Arduino, lắp đặt, code backend, update firmware OTA, viết báo cáo.
B22DCCN397	Trần Quang Huy	Lập trình Arduino, lắp đặt, code frontend, viết báo cáo.
B22DCCN398	Trần Quang Huy	Xác định yêu cầu chức năng và phi chức năng, mua linh kiện.

I. GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh và tính cấp thiết

Rò rỉ khí gas (LPG, Methan...) và nguy cơ hỏa hoạn là những mối đe dọa thường trực đối với an toàn tại các hộ gia đình, nhà xưởng, và khu vực kinh doanh. Các sự cố này không chỉ gây thiệt hại về tài sản mà còn đe dọa nghiêm trọng đến tính mạng con người do ngạt khí, cháy nổ.

Tính cấp thiết của việc phòng chống rủi ro cháy nổ và rò rỉ khí gas ngày càng gia tăng, đặc biệt tại các đô thị lớn như Hà Nội. Trong giai đoạn năm 2024 và 2025, thủ đô đã ghi nhận nhiều vụ cháy nổ nghiêm trọng xảy ra tại các khu dân cư và nhà trọ, gây ra những tổn thất lớn về người và của. Các vụ việc này, điển hình là các sự cố liên quan đến chập điện hoặc rò rỉ khí gas, đã gióng lên hồi chuông cảnh báo về sự cần thiết phải có các hệ thống giám sát và cảnh báo tự động, kịp thời để bảo vệ an toàn cho cộng đồng.

Để giải quyết vấn đề này, đề tài tập trung xây dựng một Hệ thống Phát hiện Rò rỉ Khí Gas và Cháy nổ thông minh dựa trên nền tảng Internet of Things (IoT), nhằm giám sát môi trường liên tục và tự động thực hiện các hành động khẩn cấp để giảm thiểu tối đa rủi ro.

1.2. Giới thiệu về hệ thống phát hiện rò rỉ khí gas

Để đáp ứng yêu cầu cấp thiết về an toàn phòng chống cháy nổ, đặc biệt là rò rỉ khí gas, chúng em chọn phát triển một Hệ thống phát hiện rò rỉ khí gas và Cháy nổ thông minh được xây dựng toàn diện trên nền tảng Internet of Things (IoT). Hệ thống này đại diện cho một giải pháp giám sát an ninh chủ động, mang lại khả năng theo dõi và ứng phó tức thời với các mối nguy hiểm tiềm ẩn.

Trái tim của toàn bộ hệ thống là module vi điều khiển hiệu năng cao ESP32 (cụ thể là ESP-WROOM-32S), đóng vai trò là trung tâm xử lý và điều khiển thông minh. Nhờ vào khả năng xử lý mạnh mẽ và tích hợp sẵn kết nối Wi-Fi, ESP32 cho phép hệ thống thực hiện đồng thời nhiều chức năng cốt lõi và phức tạp:

1. Giám sát Môi trường liên tục và chính xác: Hệ thống được trang bị các cảm biến chuyên dụng để liên tục theo dõi môi trường.

- Cảm biến Khí Gas MQ-2: Đây là thành phần quan trọng để theo dõi nồng độ các loại khí dễ cháy (LPG, Methan, Propane, Hydrogen) trong không khí. Hệ thống sẽ liên tục phân tích tín hiệu từ MQ-2 để xác định mức độ nguy hiểm của rò rỉ khí gas.
- Cảm biến Lửa (Flame Sensor): Được sử dụng để nhanh chóng phát hiện bức xạ hồng ngoại phát ra từ nguồn lửa, cho phép hệ thống nhận biết các sự cố hỏa hoạn đang bắt đầu.

2. Tự động hóa ứng phó khẩn cấp tại chỗ: Khi một ngưỡng nguy hiểm (rò rỉ khí gas vượt mức cho phép hoặc phát hiện ngọn lửa) được xác định, hệ thống sẽ tự động kích hoạt một loạt các thiết bị chấp hành để thực hiện các hành động khẩn cấp nhằm giảm thiểu rủi ro ngay lập tức:

- Còi báo động (Buzzer) và Đèn cảnh báo (LED): Phát ra tín hiệu âm thanh và ánh sáng cường độ cao để cảnh báo người dân và những người xung quanh về mối nguy hiểm.

- Module Relay: Được sử dụng để điều khiển các thiết bị điện công suất lớn như quạt thông gió tự động bật hoặc các van ngắt điện/gas chính để ngăn chặn việc cung cấp nhiên liệu cho đám cháy/rò rỉ.

- Động cơ Servo: Có thể được lập trình để thực hiện các hành động cơ học quan trọng, ví dụ như tự động đóng/mở van gas, hoặc mở một cửa sổ/lỗ thông gió khẩn cấp.

3. Cảnh báo và giám sát từ xa thông qua IoT: Đây là tính năng then chốt của giải pháp IoT. Hệ thống sử dụng kết nối WiFi (tần số 2.4GHz) và tận dụng nền tảng máy chủ cũng như thư viện Blynk nổi tiếng. Điều này cho phép hệ thống:

- Gửi dữ liệu thời gian thực: Mọi thông số về nồng độ khí gas và trạng thái lửa sẽ được cập nhật liên tục lên máy chủ Blynk.

- Cung cấp biểu đồ theo dõi: Người dùng có thể xem lại lịch sử và xu hướng nồng độ khí gas theo thời gian, giúp đánh giá tình trạng an toàn của môi trường sống/làm việc.

- Cảnh báo tức thời (Push Notification): Khi sự cố xảy ra (rò rỉ khí gas hoặc phát hiện lửa), hệ thống sẽ gửi cảnh báo tức thời dưới dạng thông báo đẩy (push notification) đến thiết bị di động của người dùng (smartphone, tablet), cho phép họ nhận biết và đưa ra hành động can thiệp dù đang ở bất cứ đâu.

Tóm lại, hệ thống IoT này không chỉ dừng lại ở việc phát hiện mà còn cung cấp một cơ chế phản ứng tự động, kết hợp với khả năng giám sát từ xa mạnh mẽ, tạo nên một lớp bảo vệ chủ động và hiệu quả cao trước các nguy cơ cháy nổ và rò rỉ khí gas.

1.3. Cấu trúc tổng quan

Hệ thống được thiết kế một cách khoa học và toàn diện, chủ yếu dưới dạng một mô hình học tập và nghiên cứu (study/research model). Mục tiêu của việc xây dựng mô hình là nhằm minh họa rõ ràng và kiểm chứng tính khả thi của một quy trình an toàn tự động hoàn chỉnh, từ khâu giám sát đến hành động ứng phó khẩn cấp.

Cấu trúc cốt lõi của hệ thống xoay quanh việc sử dụng vi điều khiển Arduino Nano làm trung tâm xử lý dữ liệu và điều phối hoạt động. Mặc dù ở phần giới thiệu, hệ thống đã đề cập đến nền tảng ESP32 để kết nối IoT, việc sử dụng Arduino Nano trong mô hình học tập/nghiên cứu này giúp đơn giản hóa mạch xử lý, dễ dàng lập

trình và tích hợp các module cảm biến và chấp hành một cách linh hoạt, hiệu quả nhất.

Hệ thống tích hợp đầy đủ các thành phần cần thiết để mô phỏng một quy trình an toàn tự động khép kín:

1. Module cảm biến (Sensing Modules): Đóng vai trò là "mắt" và "mũi" của hệ thống, bao gồm:

- Cảm biến Khí Gas MQ-2: Để đo lường và xác định nồng độ các loại khí dễ cháy trong môi trường.

- Cảm biến Lửa (Flame Sensor): Để phát hiện các tín hiệu nhiệt và ánh sáng đặc trưng của đám cháy.

2. Thiết bị chấp hành (Actuator Devices): Đây là các cơ cấu thực hiện hành động can thiệp khi phát hiện nguy cơ, bao gồm:

- Còi báo động (Buzzer) và Đèn Cảnh báo (LED): Phát tín hiệu khẩn cấp bằng âm thanh và ánh sáng.

- Động cơ Servo: Mô phỏng việc đóng/mở van gas tự động.

- Quạt (Fan): Mô phỏng hệ thống thông gió khẩn cấp để hút khí gas độc hại hoặc khí nóng ra khỏi khu vực.

- Bơm (Pump): Có thể được tích hợp để mô phỏng hệ thống phun nước/chất chữa cháy tự động (tùy theo mục đích mở rộng của mô hình).

Về mặt kỹ thuật, hệ thống có cấu trúc phần cứng nhỏ gọn, tối ưu cho việc trình diễn và kiểm tra trong môi trường phòng thí nghiệm hoặc mô hình thu nhỏ. Toàn bộ hệ thống hoạt động ổn định ở điện áp 5VDC và có công suất tổng tiêu thụ thấp, khoảng 5W, đảm bảo tính an toàn, tiết kiệm năng lượng và dễ dàng triển khai trong các ứng dụng thực tế. Sự kết hợp giữa khả năng xử lý của Arduino Nano và các thiết bị ngoại vi đã tạo ra một hệ thống tự động, có khả năng phản ứng nhanh chóng, minh họa rõ ràng nguyên lý hoạt động của một giải pháp phòng chống cháy nổ thông minh trong bối cảnh IoT.

II. CÔNG NGHỆ SỬ DỤNG

2.1 Bảng tổng hợp các thành phần công nghệ chính

Nhóm	Công nghệ / Thư viện cụ thể	Mục đích & vai trò trong hệ thống
Vi điều khiển	ESP32 (ESP-WROOM-32S) – Dual-core Xtensa LX6 240 MHz, 520 KB SRAM, 4–16 MB Flash, Wi-Fi 802.11 b/g/n	Bộ não trung tâm: xử lý cảm biến, điều khiển actuator, kết nối mạng, chạy FreeRTOS multitasking, hỗ trợ OTA
Ngôn ngữ lập trình thiết bị	Arduino Core for ESP32 (C/C++) + PlatformIO	Viết firmware dễ dàng, quản lý thư viện, hỗ trợ OTA native
Cảm biến	MQ-2 (khí dễ cháy & khói), Flame Sensor (hồng ngoại), DHT22 (nhiệt độ & độ ẩm – tùy chọn)	Thu thập dữ liệu môi trường thời gian thực
Thiết bị chấp hành	SG90 Servo x2, Relay 2 kênh 5V-10A, Buzzer active, LED RGB, quạt DC 5–12V	Thực hiện hành động tại chỗ: đóng van, bật quạt, báo động âm thanh – ánh sáng
Giao thức IoT chính	**MQTT 3.1.1**	Giao tiếp chính giữa ESP32 ↔ Backend: nhẹ, ổn định trong mạng yếu, tiết kiệm băng thông, hỗ trợ LWT
MQTT Broker	**Aedes** (Node.js) – broker cực nhẹ, hỗ trợ hàng nghìn kết nối đồng thời	Chạy cùng backend, xử lý toàn bộ message MQTT từ thiết bị và lệnh từ server
Backend	Node.js v20	Cung cấp REST API + xử lý logic nghiệp vụ, quản lý thiết bị, OTA firmware
Realtime từ Backend → Frontend	**WebSocket** (socket.io hoặc ws)	Đẩy dữ liệu cảm biến, cảnh báo, trạng thái thiết bị lên dashboard với độ trễ < 200 ms
Upload & quản lý OTA	upload file .bin	Cho phép người dùng upload firmware mới → server lưu → gửi lệnh OTA đến ESP32
Frontend / Dashboard	HTML5 + **JavaScript thuần**	Giao diện nhẹ, không phụ thuộc framework nặng (React/Vue), dễ bloat), cập nhật DOM trực tiếp, hiệu năng cao
Realtime trên Frontend	WebSocket	Nhận và hiển thị tức thì telemetry, alert, trạng thái online/offline

Triển khai	Chạy thủ công trên Localhost	Dễ triển khai, bảo trì và mở rộng

2.2. Vai trò chi tiết của MQTT trong hệ thống

Giao thức MQTT (Message Queuing Telemetry Transport) phiên bản 3.1.1 được chọn làm giao thức truyền thông cốt lõi giữa các thiết bị nhúng ESP32 và máy chủ trung tâm. Đây là lựa chọn tối ưu cho các ứng dụng IoT yêu cầu độ tin cậy cao, tài nguyên hạn chế và môi trường mạng không ổn định.

- Đặc điểm hoạt động

- MQTT hoạt động theo mô hình **Publish/Subscribe** thông qua một MQTT Broker trung gian (Aedes), giúp tách rời hoàn toàn thiết bị đầu cuối và ứng dụng người dùng, giảm tải kết nối trực tiếp và tối ưu băng thông.
- Hỗ trợ ba mức **Quality of Service (QoS)**, trong đó hệ thống sử dụng chủ yếu **QoS 1** (At least once) cho các gói tin cảnh báo và telemetry quan trọng, đảm bảo tin nhắn được gửi đến đúng một lần dù mạng có gián đoạn tạm thời.

- Các loại dữ liệu truyền qua MQTT

Loại dữ liệu	**Nội dung chính**
Telemetry	gas_ppm, temperature (°C), humidity (%), battery (%), rssi (dBm)
Alert	{level: "LOW"
Status	{firmware_version, uptime_s, ip, online} + LWT
Config/Command	JSON cấu hình hoặc lệnh điều khiển

- Ưu điểm nổi bật khi áp dụng cho hệ thống phát hiện khí gas & cháy

- Đảm bảo cảnh báo đến server 100% ngay cả khi mạng Wi-Fi chập chờn nhờ cơ chế retry của QoS 1.
- Phát hiện thiết bị mất kết nối trong vòng < 5 giây nhờ Last Will & Testament.
- Tiêu thụ tài nguyên cực thấp: broker Aedes chỉ chiếm khoảng 30–50 MB RAM khi phục vụ hàng trăm thiết bị đồng thời.
- Khả năng mở rộng không giới hạn chỉ bằng cách thay đổi tiền tố topic (ví dụ: building/A/floor/3/room/301/...).

2.3. Vai trò chi tiết của WebSocket trong hệ thống

WebSocket được triển khai như kênh giao tiếp thời gian thực hai chiều giữa Backend (Node.js) và Dashboard web, thay thế hoàn toàn cơ chế polling truyền thống.

- Chức năng chính

- Đẩy tức thì mọi dữ liệu từ MQTT Broker lên giao diện người dùng mà không cần người dùng refresh trang.
- Hỗ trợ truyền song công (full-duplex): người dùng có thể gửi lệnh điều khiển hoặc xác nhận cảnh báo (ACK) về server ngay lập tức.
-

- Các loại dữ liệu truyền qua WebSocket

Loại dữ liệu	**Nội dung**	**Đặc điểm nổi bật**
Telemetry	Giá trị gas, nhiệt độ, độ ẩm, biểu đồ cập	Độ trễ trung bình < 200 ms

realtime	nhật liên tục	
Cảnh báo khẩn cấp	Thông báo rò rỉ/ cháy kèm âm thanh trên trình duyệt	Hiển thị popup + rung + âm thanh ngay lập tức
Trạng thái thiết bị	Online / Offline / Low battery / Firmware version	Cập nhật tự động nhờ LWT
Log hệ thống	Các sự kiện điều khiển, OTA, lỗi kết nối	Dùng để audit và debug
Phản hồi lệnh	Kết quả thực thi lệnh (success / failed)	Người dùng biết ngay lệnh đã được thực hiện

III. PHÂN TÍCH YÊU CẦU - TÍNH NĂNG

3.1. Yêu cầu chức năng

Các yêu cầu sau đây xác định tính năng và trải nghiệm sử dụng mô hình. Chúng tập trung vào khả năng phát hiện khí gas (MQ2), kích hoạt báo động cục bộ (Must-have) và các tính năng giám sát, hiển thị từ xa/tại chỗ (Should-have).

Yêu cầu	Chi tiết (Ưu tiên)	Dựa trên tính năng
Phát hiện/Báo động	Phát hiện khí gas (MQ2) và kích hoạt báo động cục bộ (Còi/Đèn, Servo/Quạt) (Must-have).	Cảm biến MQ2 (GPIO 34), Còi/Đèn (GPIO 23), Servo (GPIO 18, 19), Relay (GPIO 25, 26).
Giám sát từ xa	Gửi thông báo thời gian thực qua app Blynk (Should-have).	Kết nối WiFi, App Blynk hiển thị biểu đồ và thông báo đẩy.
Hiển thị tại chỗ	Màn hình LCD hiển thị dữ liệu thời gian thực và trạng thái (Should-have).	Màn hình LCD qua I2C (SDA: GPIO 21, SCL: GPIO 22).
Cài đặt	Có thể sử dụng nút nhấn để reset hoặc điều chỉnh ngưỡng báo động.	4 nút nhấn 6x6mm .

3.2. Yêu cầu kỹ thuật

Các yêu cầu về mặt kỹ thuật dưới đây giúp đảm bảo hiệu suất và ổn định của hệ thống. Yêu cầu cốt lõi là sử dụng nền tảng ESP32-WROOM-32S và phải thực hiện tối ưu hóa bộ nhớ cùng với đảm bảo kết nối Wi-Fi ổn định và áp dụng các biện pháp bảo mật dữ liệu cho giao tiếp Blynk.

Yêu cầu	Chi tiết	Dựa trên Thiết kế
Nền tảng	Sử dụng KIT ESP32-WROOM-32S, CPU dual-core 240MHz.	Đã chốt nền tảng.

Bộ nhớ	Cần tối ưu hóa code và quản lý bộ nhớ (RAM 520KB, Flash 4MB) để tránh tràn bộ nhớ khi xử lý WiFi và cảm biến.	Sử dụng buffer nhỏ cho dữ liệu analog, loại bỏ thư viện thừa.
Kết nối	Sử dụng WiFi 2.4GHz ổn định.	Sử dụng module WiFi tích hợp của ESP32 (ESP-WROOM-32S).
Bảo mật	Dữ liệu nhạy cảm (biểu đồ) phải tuân thủ Luật An ninh mạng.	Sử dụng HTTPS cho kết nối Blynk; tích hợp xác thực hai lớp.

3.3. Các tính năng triển khai

● **Phát hiện và giám sát nồng độ khí gas trong không khí**

- Hệ thống cần được trang bị cảm biến MQ-2 để có thể phát hiện và đo lường nồng độ khí gas (chẳng hạn như LPG, butan, propane, methane, khói, CO, v.v.) trong môi trường xung quanh. Cảm biến này liên tục thu thập dữ liệu và gửi tín hiệu về vi điều khiển ESP32 để xử lý.
- Khi nồng độ khí gas được phát hiện vượt quá một giá trị giới hạn an toàn (ngưỡng gas được đặt trước), hệ thống phải tự động kích hoạt cơ chế cảnh báo nhằm thông báo cho người dùng biết tình trạng nguy hiểm, tránh xảy ra cháy nổ hoặc ngộ độc khí.

● **Cảnh báo khi xảy ra rò rỉ khí gas**

- Khi giá trị nồng độ khí gas đo được vượt ngưỡng an toàn, hệ thống sẽ kích hoạt còi báo cháy hoặc thiết bị phát âm thanh cảnh báo tại chỗ (còi báo động, đèn cảnh báo) giúp người ở gần nhận biết ngay lập tức.
- Bên cạnh đó, hệ thống còn gửi thông báo cảnh báo đến người dùng từ xa thông qua mạng Internet hoặc ứng dụng web.
- Tùy thuộc vào mức độ hoàn thiện của hệ thống, có thể mở rộng thêm các tính năng điều khiển tự động, ví dụ như:

+ Kích hoạt quạt thông gió để khuếch tán và làm loãng nồng độ khí trong không khí.
+ Kích hoạt máy bơm hoặc hệ thống phun nước để giảm rủi ro cháy nổ.
+ Đóng hoặc mở cửa tự động để thông gió hoặc cô lập khu vực rò rỉ.

Những chức năng này góp phần nâng cao tính an toàn và tự động hóa của toàn hệ thống.

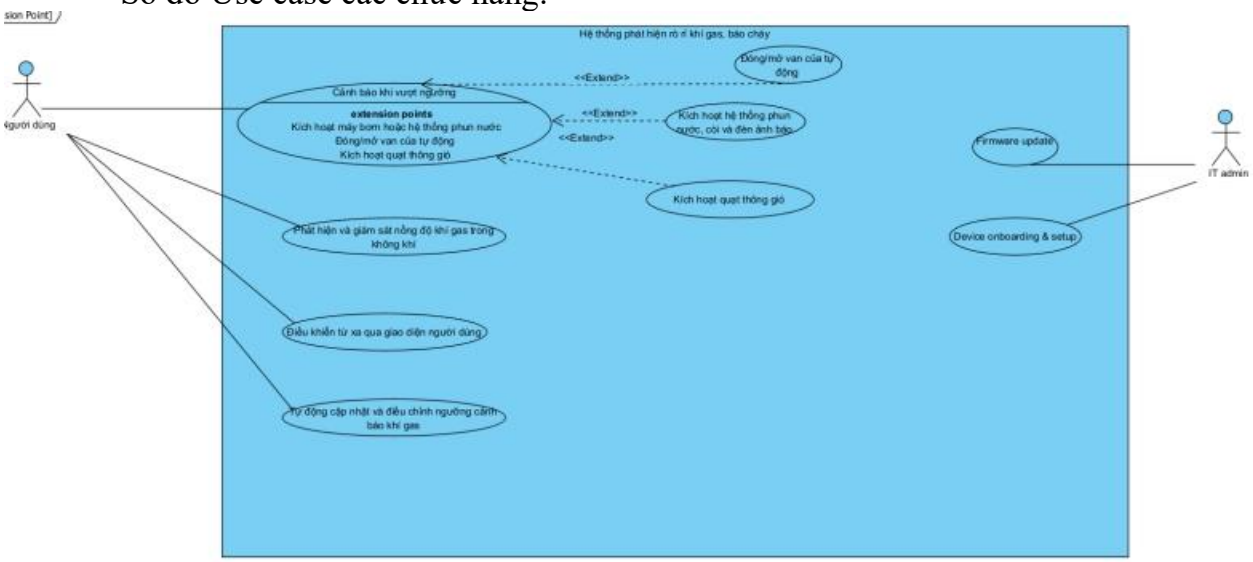
● **Điều khiển từ xa qua giao diện người dùng**

- Dashboard web được xây dựng bằng JavaScript, hoạt động mượt mà trên mọi thiết bị (máy tính, điện thoại, tablet).
- Người dùng có thể:
  + Theo dõi realtime toàn bộ thông số môi trường và trạng thái cảm biến.
  + Quan sát trạng thái hiện tại của các thiết bị chấp hành (quạt thông gió, van gas servo, relay, còi báo động, đèn cảnh báo…).
  + Thực hiện điều khiển thủ công từ xa: bật/tắt quạt, mở/đóng van, kích hoạt còi kiểm tra, reboot thiết bị, v.v.
- Mọi thao tác được truyền qua WebSocket và MQTT, phản hồi gần như tức thì (thường dưới 500 ms).

● **Tự động cập nhật và điều chỉnh ngưỡng cảnh báo khí gas**

- Người dùng có thể thay đổi linh hoạt các ngưỡng cảnh báo (thấp – cao – nguy cấp) và chu kỳ gửi dữ liệu trực tiếp trên giao diện dashboard.
- Sau khi lưu, backend ngay lập tức publish giá trị mới xuống topic config tương ứng.
- ESP32 nhận lệnh, cập nhật ngưỡng vào bộ nhớ không mất và áp dụng ngay lập tức cho các lần đo tiếp theo, đồng thời gửi xác nhận về dashboard.
- Tính năng này cho phép tùy chỉnh độ nhạy của hệ thống phù hợp với từng môi trường cụ thể như bếp gia đình, nhà kho, nhà xưởng hoặc khu vực sản xuất.

- Sơ đồ Use case các chức năng:



3.4. Các yêu cầu phi chức năng

- **Độ tin cậy & An toàn**

- Hoạt động độc lập : Cảnh báo tại chỗ (Còi) phải hoạt động ngay cả khi mất kết nối WiFi/Server. Đây là ưu tiên số 1 của hệ thống an toàn.
- Tự phục hồi: Thiết bị (ESP32) phải có cơ chế tự động kết nối lại (Auto-reconnect) khi mạng chập chờn.

- **Hiệu suất hệ thống**

- Hệ thống phải đảm bảo phản hồi nhanh và chính xác khi phát hiện khí gas vượt ngưỡng.Thời gian phản hồi giữa cảm biến và còi báo không vượt quá 1 giây, đảm bảo cảnh báo kịp thời trong tình huống khẩn cấp.
- Toàn bộ quá trình xử lý dữ liệu cần diễn ra ổn định, liên tục mà không bị trễ hoặc mất tín hiệu.

- **Khả năng mở rộng**

- Hệ thống được thiết kế theo hướng mở và linh hoạt, cho phép bổ sung thêm nhiều loại cảm biến khác nhau như cảm biến nhiệt độ, độ ẩm, phát hiện lửa hoặc khói trong tương lai.
- Ngoài ra, có thể mở rộng kết nối với nhiều thiết bị điều khiển khác như quạt, máy bơm, hoặc cửa tự động mà không cần thay đổi cấu trúc chính.

- **Khả năng bảo trì và nâng cấp**

- Hệ thống phải có thiết kế dễ bảo trì và mở rộng, cho phép cập nhật phần mềm hoặc thay đổi phần cứng mà không ảnh hưởng đến hoạt động chung.
- Cấu trúc module hóa giúp thuận tiện cho việc sửa chữa, nâng cấp firmware, hoặc bổ sung tính năng mới trong tương lai.

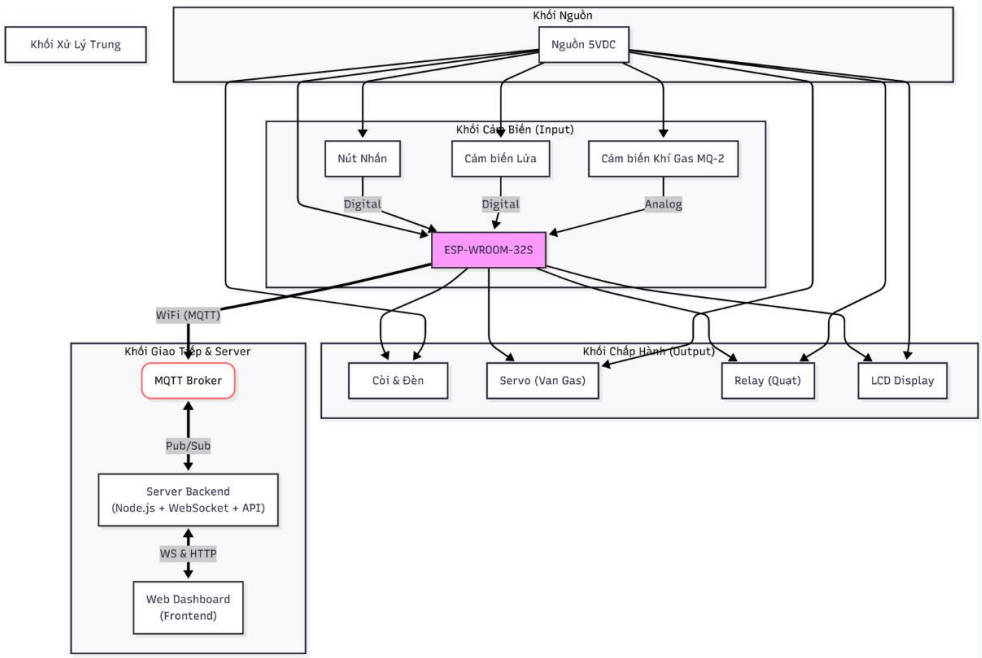
- **Tính khả dụng**

- Thiết bị phải dễ sử dụng, dễ lắp đặt và phù hợp với nhiều môi trường như nhà ở, lớp học, phòng thí nghiệm, kho chứa hoặc mô hình giảng dạy.
- Thiết kế nhỏ gọn, bố trí hợp lý, đảm bảo tính thẩm mỹ và không gây cản trở không gian sử dụng.
- Vỏ thiết bị cần có độ bền cao, chịu nhiệt tốt và không làm ảnh hưởng đến độ nhạy của cảm biến.

IV. PHÂN TÍCH THIẾT KẾ

4.1. Kiến trúc hệ thống

Hệ thống được thiết kế theo các khối, chia thành các tầng chức năng độc lập nhằm dễ phát triển, dễ mở rộng và dễ bảo trì. Sơ đồ kiến trúc thể hiện các luồng xử lý giữa khối nguồn, khối cảm biến, khối xử lý trung tâm, khối chấp hành, và khối giao tiếp – server.



Thiết kế tổng quan hệ thống

4.1.1. Khối Nguồn

Khối nguồn cung cấp năng lượng cho toàn bộ hệ thống:

- Nguồn 5VDC cấp cho:
  - ESP32-WROOM-32S
  - Các cảm biến (MQ-2, Flame)
  - LCD
  - Module Relay
  - Servo SG90
  - Buzzer/đèn cảnh báo

Nguồn được chọn đảm bảo:

- Dòng tải tối thiểu 2A để đáp ứng dòng khởi động của MQ-2 và Servo.
- Ổn định áp khi toàn hệ thống hoạt động đồng thời.

Khối nguồn đóng vai trò nền tảng duy trì tính ổn định của toàn bộ hệ thống.

4.1.2. Khối Cảm Biến

Khối này là "giác quan" của hệ thống, thu thập dữ liệu từ môi trường và gửi về vi điều khiển:

- Cảm biến khí gas MQ-2 (Analog Input)
  - Đo nồng độ LPG/Butan/Methane.

    - Kết nối vào chân ADC (GPIO34) của ESP32.
  - Cảm biến lửa (Flame Sensor – Digital Input)
    - Phát hiện bức xạ hồng ngoại từ lửa.
    - Kết nối qua chân digital.
  - Nút nhấn (Button Input)
    - Dùng để reset, cài đặt lại ngưỡng hoặc kích hoạt các cơ chế thủ công.

Mỗi tín hiệu từ cảm biến được đưa trực tiếp vào ESP32 để xử lý theo thời gian thực.

4.1.3. Khối Xử Lý Trung Tâm

ESP32 đóng vai trò trung tâm điều khiển:

- Thu thập dữ liệu từ MQ-2, Flame Sensor, Button.
- Xử lý logic an toàn, bao gồm:
    - So sánh nồng độ khí với ngưỡng cảnh báo.
    - Kiểm tra trạng thái lửa.
    - Kích hoạt thiết bị chấp hành khi có nguy cơ.
- Điều khiển relay, servo, buzzer, LCD.
- Giao tiếp qua WiFi với MQTT Broker.
- Gửi/nhận dữ liệu với backend và Web Dashboard.
- Thực hiện OTA Firmware Update khi có yêu cầu.

ESP32 đóng vai trò bridge giữa phần cứng và tầng server/backend.

4.1.4. Khối Chấp Hành

Đây là nơi thực hiện các hành động phản ứng:

- Còi & Đèn cảnh báo
    - Phát tín hiệu khi vượt ngưỡng gas hoặc phát hiện lửa.
- Servo (Van Gas)
    - Mô phỏng cơ chế đóng/mở van gas an toàn.
- Relay (Quạt thông gió)
    - Kích hoạt quạt hút để làm loãng khí gas trong phòng.
- LCD hiển thị
    - Hiển thị các giá trị MQ-2, trạng thái cảnh báo, thông tin hệ thống.

Khối chấp hành cho phép hệ thống phản ứng ngay lập tức khi phát hiện nguy cơ.

4.1.5. Khối Giao Tiếp & Server

Khối này chịu trách nhiệm tạo giao diện và xử lý dữ liệu từ xa:

a. MQTT Broker

- Nhận dữ liệu từ ESP32 qua WiFi.
- Pub/Sub để phân phối dữ liệu đến Backend.
- Giữ vai trò trung gian truyền thông theo mô hình IoT chuẩn.

b. Server Backend (Node.js + WebSocket + REST API)

- Nhận dữ liệu từ MQTT Broker.
- Xử lý, lưu trữ, và gửi dữ liệu realtime đến Web Dashboard.
- Gửi lệnh điều khiển trở lại ESP32 thông qua MQTT.

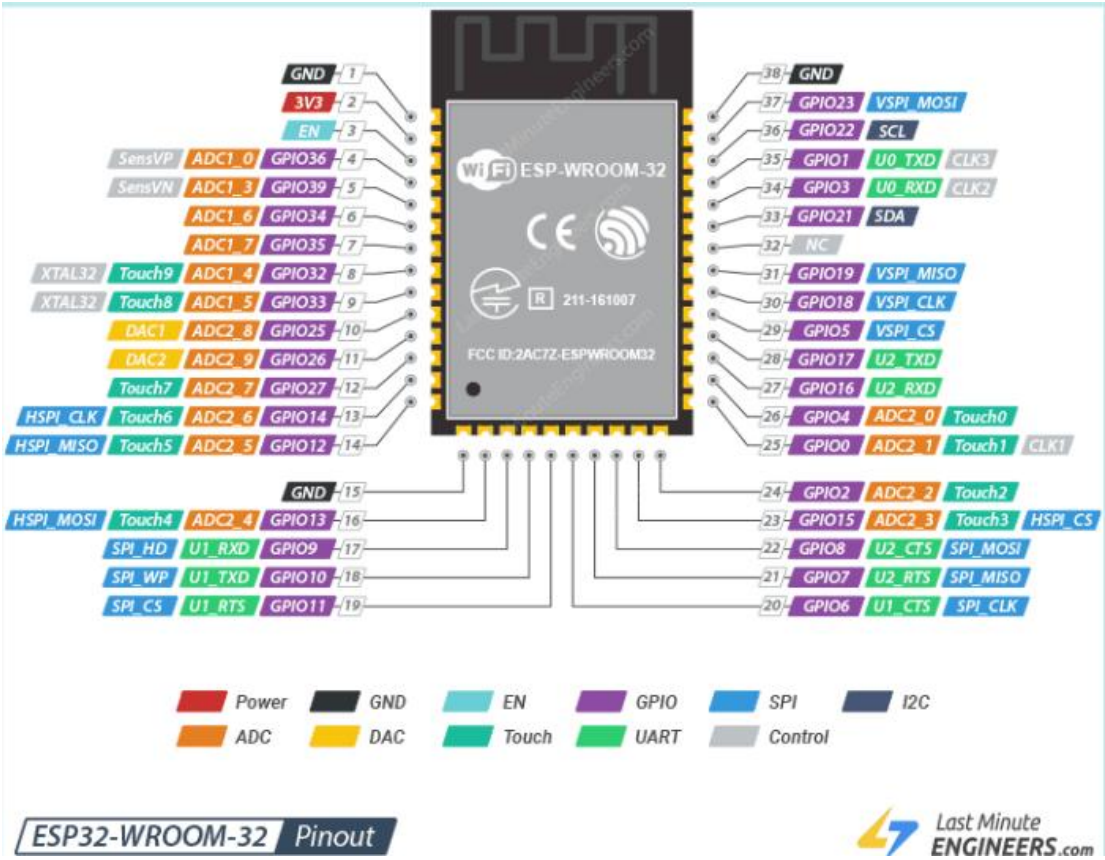
Backend đóng vai trò xử lý logic mở rộng và làm trung gian giữa giao diện người dùng và thiết bị.

c. Web Dashboard (Frontend)

- Hiển thị các dữ liệu:
  - Nồng độ khí gas
  - Trạng thái lửa
  - Trạng thái thiết bị (servo, relay)
- Điều khiển thiết bị từ xa qua WebSocket/API.
- Theo dõi realtime, không cần cài app.

4.2. Thiết kế giao tiếp phần cứng

Thiết kế giao tiếp phần cứng được xây dựng dựa trên khả năng của vi điều khiển ESP-WROOM-32S, kết hợp với các cảm biến và thiết bị chấp hành trong hệ thống. Mỗi linh kiện được kết nối với ESP32 thông qua các chuẩn giao tiếp phù hợp (Analog, Digital, PWM, I2C).



Sơ đồ các chân kết nối ESP32

4.2.1. Bảng mô tả giao tiếp phần cứng

Linh kiện	Chân kết nối với ESP32	Chức năng

Cảm biến khí gas MQ2	Chân analog (GPIO 34)	Đọc tín hiệu analog (0–4095) để đo nồng độ khí gas trong môi trường.
Cảm biến lửa (Flame Sensor)	Chân digital (GPIO 27)	Đọc tín hiệu số (0 hoặc 1) để phát hiện ngọn lửa/bức xạ hồng ngoại.
Động cơ servo SG90 (2 cái)	Chân PWM (GPIO 18 và GPIO 19)	Xuất tín hiệu PWM để điều khiển góc quay servo (đóng/mở van gas hoặc mô phỏng cơ cấu an toàn).
Module relay 2 kênh	Chân digital (GPIO 25 và GPIO 26)	Xuất tín hiệu HIGH/LOW để bật/tắt quạt thông gió hoặc đèn cảnh báo.
Còi và đèn LED cảnh báo	Chân digital (GPIO 23)	Xuất tín hiệu điều khiển phát âm thanh và ánh sáng cảnh báo khi có sự cố.
Màn hình LCD 1602 / 2004 (I2C)	SDA → GPIO 21	
SCL → GPIO 22	Giao tiếp qua I2C để hiển thị nồng độ khí gas, trạng thái cảm biến, cảnh báo hệ thống.	

4.2.2. Giải thích từng chân

MQ2 → GPIO 34 (ADC1)

- GPIO34 thuộc nhóm ADC1, có độ ổn định cao hơn ADC2 khi WiFi hoạt động.
- MQ2 cần đọc giá trị analog liên tục → ADC1 là lựa chọn tối ưu.

Flame Sensor → GPIO 27

- Là chân digital phổ biến, hỗ trợ input ổn định.
- Tránh các chân bị hạn chế (GPIO 34–39 chỉ input, GPIO 0/2/15 dùng cho boot mode).

Servo SG90 → GPIO 18 & 19

- Hai chân này hỗ trợ PWM chuẩn.
- Không ảnh hưởng đến giao tiếp I2C/SPI/UART khác.

Relay → GPIO 25 & 26

- Hai chân này hoạt động tốt ở chế độ Digital Output.
- Khả năng chịu tải phù hợp để kích relay mức LOW/HIGH.

Còi & LED → GPIO 23

- GPIO23 là chân out mạnh, phù hợp cho tải nhỏ như buzzer/LED.

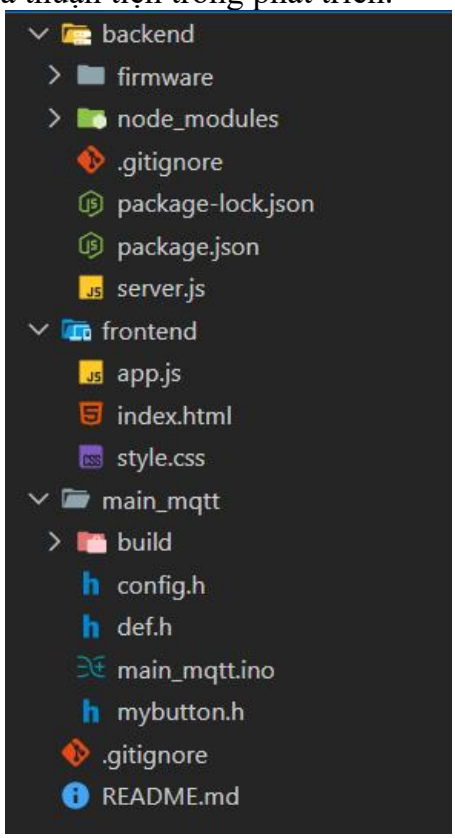
LCD I2C → GPIO 21 (SDA), 22 (SCL)

- Đây là cặp chân I2C mặc định trên ESP32.
- Tương thích với hầu hết module LCD I2C (PCF8574).

4.3. Thiết kế web điều khiển

4.3.1. Kiến trúc thư mục code

Mã nguồn được tổ chức theo cấu trúc thư mục tách biệt ba thành phần chính: backend, frontend, và main_mqtt. Mỗi thành phần đảm nhiệm một nhiệm vụ cụ thể, đảm bảo tính mở rộng, dễ bảo trì và thuận tiện trong phát triển.



Thư mục backend/
Chứa toàn bộ mã nguồn của máy chủ Web và MQTT. Thành phần này bao gồm Express server, WebSocket server và MQTT broker (Aedes). Backend có nhiệm vụ:

- Giao tiếp MQTT với ESP32 (nhận dữ liệu & gửi lệnh điều khiển).
- Xử lý WebSocket để cập nhật realtime cho giao diện Web.
- Cung cấp REST API cho quản lý thiết bị và OTA.
- Lưu và phục vụ các file firmware để ESP32 thực hiện OTA Update.

Bên trong backend/ có thư mục firmware/ dùng để lưu các file .bin cùng metadata.

Thư mục frontend/
Là giao diện người dùng hiển thị trên trình duyệt. Frontend gồm ba tệp chính:

- index.html: bố cục giao diện Dashboard.
- style.css: các quy tắc CSS, màu sắc cảnh báo, bố cục responsive.
- app.js: toàn bộ logic WebSocket, gọi API, cập nhật UI và gửi lệnh điều khiển.

Frontend kết nối trực tiếp với backend bằng WebSocket để nhận dữ liệu cảm biến thời gian thực và phản hồi trạng thái thiết bị.

Thư mục main_mqtt/
Chứa mã nguồn điều khiển ESP32:

- Đọc cảm biến MQ2 và Flame.
- Publish dữ liệu lên MQTT broker.
- Subscribe các topic điều khiển để vận hành relay, servo, buzzer.
- Nhận lệnh OTA từ backend và tải firmware mới.

4.3.2. Thiết kế Frontend

a. Cơ chế hoạt động của Frontend

Kết nối WebSocket realtime

Frontend tạo kết nối WebSocket:

```
// WebSocket connection
const WS_URL = 'ws://localhost:8888';
const API_URL = 'http://localhost:3000/api';

let ws = null;
let currentDeviceId = null;
let deviceStatus = {};

// Initialize WebSocket connection
function initWebSocket() {
    ws = new WebSocket(WS_URL);

    ws.onopen = () => {
        console.log('WebSocket connected');
        updateConnectionStatus(true);
        showAlert('Kết nối thành công!', 'success');
    };

    ws.onmessage = (event) => {
        const data = JSON.parse(event.data);
        handleWebSocketMessage(data);
    };

    ws.onerror = (error) => {
        console.error('WebSocket error:', error);
        updateConnectionStatus(false);
    };

    ws.onclose = () => {
        console.log('WebSocket disconnected');
        updateConnectionStatus(false);
        // Reconnect after 3 seconds
        setTimeout(initWebSocket, 3000);
    };
}
```

WebSocket dùng cho:

- Nhận dữ liệu realtime từ thiết bị (gas, lửa, relay, servo)
- Nhận trạng thái OTA
- Nhận danh sách thiết bị đang online
- Gửi lệnh điều khiển xuống backend

Sau khi kết nối, dashboard sẽ:

- Hiển thị trạng thái "Đã kết nối"
- Nhận danh sách thiết bị (device_list)
- Lắng nghe mọi cập nhật từ backend

Cơ chế tự kết nối lại

Nếu WebSocket bị mất kết nối → tự động kết nối lại sau 3 giây.

b. Xử lý các message WebSocket

Frontend sử dụng handleWebSocketMessage() để phân loại dữ liệu nhận từ backend:

```
function handleWebSocketMessage(data) {
    console.log('WebSocket message received:', data);

    switch (data.type) {
        case 'device_list':
            updateDeviceList(data.devices);
            break;
        case 'device_data':
            if (data.deviceId === currentDeviceId) {
                updateDeviceData(data.data);
            }
            break;
        case 'device_disconnected':
            if (data.deviceId === currentDeviceId) {
                showAlert('Thiết bị đã ngắt kết nối', 'warning');
            }
            break;
        case 'ota_status':
            console.log('OTA status received:', data, 'currentDeviceId:', currentDeviceId);
            if (data.deviceId === currentDeviceId) {
                handleOTAStatus(data);
            } else {
                console.log('OTA status for different device. Expected:', currentDeviceId, 'Got:', data.deviceId);
            }
            break;
        case 'error':
            showAlert(data.message, 'error');
            break;
    }
}
```

Các loại dữ liệu quan trọng:

Kiểu message	Ý nghĩa
device_list	Danh sách thiết bị đang online
device_data	Dữ liệu realtime từ ESP32 (gas, flame, relay, servo)
device_disconnected	ESP32 bị ngắt kết nối
ota_status	Trạng thái cập nhật OTA
error	Báo lỗi từ server

Thông tin này được dùng để cập nhật UI theo thời gian thực.

c. Hiển thị dữ liệu cảm biến và trạng thái thiết bị

Frontend cập nhật UI bằng hàm:
updateDeviceData(data)

```
function updateDeviceData(data) {
    deviceStatus = data;

    // Update sensor values
    if (data.gasValue !== undefined) {
        document.getElementById('gasValue').textContent = `${data.gasValue} ppm`;
        updateGasProgress(data.gasValue, data.threshold || 4000);
    }

    if (data.fireValue !== undefined) {
        document.getElementById('fireValue').textContent = data.fireValue ? 'CÓ LỬA' : 'KHÔNG CÓ LỬA';
        document.getElementById('fireValue').style.color = data.fireValue ? '#f44336' : '#4CAF50';
    }

    // Update threshold
    if (data.threshold !== undefined) {
        document.getElementById('thresholdInput').value = data.threshold;
    }

    // Update relay states
    if (data.relay1State !== undefined) {
        const btn1 = document.getElementById('relay1Btn');
        btn1.textContent = data.relay1State ? 'BẬT' : 'TẮT';
        btn1.className = `btn-toggle ${data.relay1State ? 'on' : 'off'}`;
    }

    if (data.relay2State !== undefined) {
        const btn2 = document.getElementById('relay2Btn');
        btn2.textContent = data.relay2State ? 'BẬT' : 'TẮT';
        btn2.className = `btn-toggle ${data.relay2State ? 'on' : 'off'}`;
    }

    // Update window state
    if (data.windowState !== undefined) {
        const windowBtn = document.getElementById('windowBtn');
        windowBtn.textContent = data.windowState ? 'MỞ' : 'ĐÓNG';
```

Trong đó:

- Cập nhật giá trị MQ2
    - Hiển thị số ppm
    - Cập nhật thanh progress màu
    - Đổi màu theo mức nguy hiểm (xanh → vàng → đỏ)
- Cập nhật flame sensor
    - Hiển thị "CÓ LỬA" hoặc "KHÔNG CÓ LỬA"
    - Đổi màu chữ (đỏ khi có lửa)
- Relay, servo, cửa sổ
    - Cập nhật text của nút: BẬT/TẮT hoặc MỞ/ĐÓNG
    - Đổi màu nút theo trạng thái
- Auto/Manual
    - Toggle chuyển chế độ tự động ↔ thủ công
- Hiển thị thông tin thiết bị
    - Device ID
    - IP Address
    - Thời điểm cập nhật cuối (lastUpdate)

d. Điều khiển thiết bị từ Dashboard

Các nút bật tắt relay, cửa sổ, thay đổi threshold… đều gọi chung hàm: sendControlCommand(command)

```
function sendControlCommand(command) {
    if (!currentDeviceId) {
        showAlert('Vui lòng chọn thiết bị', 'warning');
        return;
    }

    if (ws && ws.readyState === WebSocket.OPEN) {
        ws.send(JSON.stringify({
            type: 'control',
            deviceId: currentDeviceId,
            ...command
        }));
    } else {
        // Fallback to HTTP API
        fetch(`${API_URL}/devices/${currentDeviceId}/control`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(command)
        }).catch(error => {
            console.error('Error sending control:', error);
            showAlert('Lỗi gửi lệnh điều khiển', 'error');
        });
    }
}
```

Frontend gửi lệnh qua WebSocket:

```
ws.send(JSON.stringify({
   type: 'control',
   deviceId: currentDeviceId,
   relay1: ...,
   relay2: ...,
   window: ...,
   threshold: ...,
   autoManual: ...
}));
```

Nếu WS bị mất → fallback sang HTTP API.

e. Hệ thống OTA Firmware Update (phía frontend)

Phần frontend hỗ trợ OTA theo 2 bước:

B1. Upload firmware mới

Hàm:

uploadFirmware()

```javascript
async function uploadFirmware() {
    const fileInput = document.getElementById('firmwareFile');
    const versionInput = document.getElementById('firmwareVersion');

    if (!fileInput || !fileInput.files || fileInput.files.length === 0) {
        showAlert('Vui lòng chọn file firmware (.bin)', 'warning');
        return;
    }

    const file = fileInput.files[0];
    if (!file.name.endsWith('.bin')) {
        showAlert('File phải có định dạng .bin', 'error');
        return;
    }

    const formData = new FormData();
    formData.append('firmware', file);
    if (versionInput && versionInput.value.trim()) {
        formData.append('version', versionInput.value.trim());
    }

    const statusDiv = document.getElementById('otaStatus');
    if (statusDiv) {
        statusDiv.className = 'ota-status active';
        statusDiv.textContent = 'Đang upload firmware...';
    }

    try {
        const response = await fetch(`${API_URL}/firmware/upload`, {
            method: 'POST',
            body: formData
        });

        const result = await response.json();

        if (result.success) {
            if (statusDiv) {
```

Chức năng:

- Kiểm tra định dạng file .bin
- Gửi file qua POST /api/firmware/upload
- Hiển thị tiến trình upload
- Cập nhật danh sách firmware

B2. Chọn và gửi lệnh OTA xuống thiết bị

Hàm:

startOTAUpdate()

```
async function startOTAUpdate() {
    const firmwareSelect = document.getElementById('firmwareSelect');
    if (!firmwareSelect) {
        showAlert('Không tìm thấy dropdown firmware', 'error');
        return;
    }

    const version = firmwareSelect.value;
    if (!version) {
        showAlert('Vui lòng chọn phiên bản firmware', 'warning');
        return;
    }

    if (!currentDeviceId) {
        showAlert('Vui lòng chọn thiết bị', 'warning');
        return;
    }

    // Clear any existing timeout
    if (otaUpdateTimeout) {
        clearTimeout(otaUpdateTimeout);
        otaUpdateTimeout = null;
    }

    const statusDiv = document.getElementById('otaStatus');
    if (statusDiv) {
        statusDiv.className = 'ota-status active info';
        statusDiv.textContent = 'Đang gửi lệnh cập nhật...';
    }

    try {
        const response = await fetch(`${API_URL}/devices/${currentDeviceId}/ota`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ version: version })
        });
```

Frontend:

- Lấy phiên bản firmware được chọn
- Gửi POST /api/devices/{deviceId}/ota
- Render trạng thái OTA:
    - bắt đầu
    - thành công
    - thất bại
    - timeout

OTA UI được cập nhật realtime bởi WebSocket.

f. Hệ thống cảnh báo Pop-up

Frontend sử dụng hàm:
showAlert(message, type)
Loại cảnh báo:

- success (xanh)
- warning (vàng)
- error (đỏ)
- info (xanh nhạt)

Tự động ẩn sau 3 giây.

g. Quản lý nhiều thiết bị

Frontend cho phép chọn device:
updateDeviceList(devices);

UI gồm dropdown chọn thiết bị.
Khi chọn thiết bị khác → dashboard chuyển sang data của thiết bị đó ngay lập tức.

4.3.3. Thiết kế Backend

a. MQTT Broker tích hợp (Aedes)

Backend tự chạy một MQTT broker trên cổng 1883, không cần cài đặt Mosquitto bên ngoài.
Chức năng:

- Nhận data từ ESP32 qua MQTT.
- Forward dữ liệu lên Web Dashboard qua WebSocket.
- Nhận lệnh điều khiển từ Web Dashboard → publish ngược lại xuống ESP32.
- Xử lý publish của ESP32 theo topic:
  - device/<id>/status
  - device/<id>/ota/status
  - device/<id>/control
  - ...

```
aedes.on('publish', (packet, client) => {
  if (client) {
    const topic = packet.topic;
    const message = packet.payload.toString();

    try {
      const data = JSON.parse(message);

      // Xử lý OTA status
      if (topic.includes('/ota/status')) {
        const deviceId = client.id;
        console.log(`[OTA] Device ${deviceId}: ${data.status} - ${data.message}`);

        // Broadcast OTA status đến WebSocket clients
        broadcastToWebSocket({
          type: 'ota_status',
          deviceId: deviceId,
          status: data.status,
          message: data.message,
          version: data.version,
          timestamp: data.timestamp
        });
      }
      // Lưu trạng thái thiết bị
      else if (topic.startsWith('device/')) {
        const deviceId = client.id;
        deviceStatus.set(deviceId, { ...data, lastUpdate: new Date() });
```

Sự kiện chính:

- client → thông báo thiết bị kết nối
- clientDisconnect → thiết bị offline
- publish → xử lý dữ liệu cảm biến + OTA
- subscribe → theo dõi topic ESP32 đã đăng ký

Backend lưu trạng thái vào hai Map:

```
// Lưu trữ dữ liệu thiết bị
const devices = new Map();
const deviceStatus = new Map();
```

Đây là cơ chế cần thiết để quản lý nhiều thiết bị IoT cùng lúc.

b. WebSocket Server (Realtime Dashboard)

Backend khởi chạy WebSocket Server riêng trên cổng 8888.

Chức năng WebSocket:

- Khi browser kết nối: gửi danh sách toàn bộ thiết bị đang online.
- Nhận lệnh từ browser → gửi xuống ESP32 qua MQTT:
  - bật/tắt relay
  - mở/đóng cửa sổ
  - bật chế độ auto/manual
  - thay đổi threshold

```
// ==================== WebSocket Server ====================
const wss = new ws.Server({ port: WS_PORT });

wss.on('connection', (wsClient) => {
  console.log('[WebSocket] Client connected');

  // Gửi danh sách thiết bị hiện tại
  const deviceList = Array.from(deviceStatus.entries()).map(([id, data]) => ({
    deviceId: id,
    ...data
  }));
  wsClient.send(JSON.stringify({ type: 'device_list', devices: deviceList }));

  wsClient.on('message', (message) => {
    try {
      const command = JSON.parse(message);
      handleWebSocketCommand(command, wsClient);
    } catch (e) {
      console.error('[WebSocket] Error parsing message:', e);
    }
  });

  wsClient.on('close', () => {
    console.log('[WebSocket] Client disconnected');
  });
});

function handleWebSocketCommand(command, wsClient) {
```

Dữ liệu gửi về WebSocket:

```
{
  type: "device_data",
  deviceId: "esp32_xxx",
  data: {
    gas: ...,
    flame: ...,
    relay1: ...,
    relay2: ...,
    window: ...,
    autoManual: ...,
  }
}
```

Các loại message quan trọng:

- device_list
- device_data
- device_status
- ota_status
- device_disconnected
- error

Điều này giúp giao diện Web hoạt động realtime 100%, không cần polling.

c. Express REST API

Backend cung cấp API REST chuẩn để:

Quản lý thiết bị

- GET /api/devices
- GET /api/devices/:deviceId
- POST /api/devices/:deviceId/control

```
// ==================== REST API ====================

// API: Lấy danh sách thiết bị
app.get('/api/devices', (req, res) => {
  const deviceList = Array.from(deviceStatus.entries()).map(([id, data]) => ({
    deviceId: id,
    ...data
  }));
  res.json(deviceList);
});

// API: Lấy trạng thái thiết bị
app.get('/api/devices/:deviceId', (req, res) => {
  const status = deviceStatus.get(req.params.deviceId);
  if (status) {
    res.json(status);
  } else {
    res.status(404).json({ error: 'Device not found' });
  }
});

// API: Điều khiển thiết bị
app.post('/api/devices/:deviceId/control', (req, res) => {
  const { deviceId } = req.params;
  const { relay1, relay2, window, autoManual, threshold } = req.body;

  const controlTopic = `device/${deviceId}/control`;
  const controlMessage = JSON.stringify({
```

Quản lý Firmware

- Upload firmware:
  POST /api/firmware/upload
- Lấy danh sách firmware:
  GET /api/firmware
- Download firmware:
  GET /api/firmware/:version
- Gửi lệnh OTA update:
  POST /api/devices/:deviceId/ota

Các API đã có xử lý lỗi đầy đủ (404, 500), đảm bảo an toàn và rõ ràng.

d. Hệ thống cập nhật Firmware OTA

- Upload firmware
  - Dùng multer để upload file .bin
  - Tự động tạo:
    - file firmware
    - file metadata tương ứng

```
// API: Upload firmware
app.post('/api/firmware/upload', upload.single('firmware'), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ error: 'No firmware file uploaded' });
  }

  const version = req.body.version || Date.now().toString();
  const filename = req.file.filename;

  // Lưu metadata
  const metadata = {
    version: version,
    filename: filename,
    size: req.file.size,
    uploadedAt: new Date().toISOString(),
    path: path.join(firmwareDir, filename)
  };

  const metadataPath = path.join(firmwareDir, `metadata_${version}.json`);
  fs.writeJsonSync(metadataPath, metadata);

  res.json({
    success: true,
    message: 'Firmware uploaded successfully',
    metadata: metadata
  });
});
```

- Lưu trữ firmware
  - Lưu trong thư mục /backend/firmware/
  - Metadata lưu version, filename, size, path, uploadAt

- Gửi lệnh OTA xuống ESP32 qua MQTT

```
// API: Gửi lệnh OTA update
app.post('/api/devices/:deviceId/ota', (req, res) => {
  const { deviceId } = req.params;
  const { version, url } = req.body;

  if (!version && !url) {
    return res.status(400).json({ error: 'Version or URL required' });
  }
        Đỗ Đức Cảnh, last week • done code
  let firmwareUrl = url;
  if (version && !url) {
    // Use actual IP address instead of localhost so ESP32 can access it
    const serverIP = getServerIP();
    const port = PORT;
    firmwareUrl = `http://${serverIP}:${port}/api/firmware/${version}`;
    console.log(`[OTA] Firmware URL: ${firmwareUrl}`);
  }

  const otaTopic = `device/${deviceId}/ota`;
  const otaMessage = JSON.stringify({
    version: version,
    url: firmwareUrl,
    timestamp: Date.now()
  });

  aedes.publish({
    topic: otaTopic,
    payload: Buffer.from(otaMessage),
    qos: 1
  }, (err) => {
    if (err) {
      res.status(500).json({ error: 'Failed to send OTA command' });
    } else {
      res.json({ success: true, message: 'OTA update command sent', url: firmwareUrl });
    }
  });
});
```

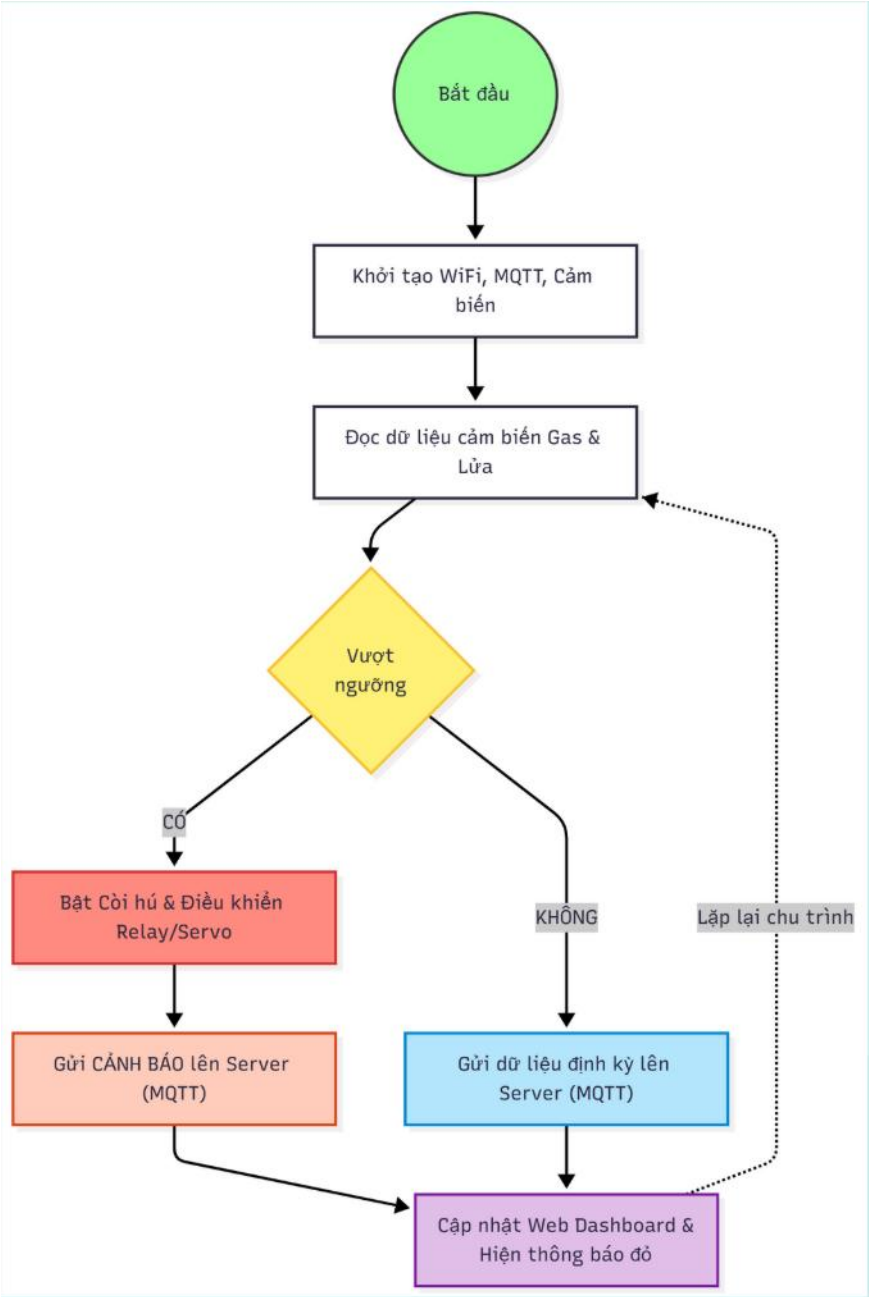
Topic:
device/<deviceId>/ota
Payload:
{
 "version": "...",
 "url": "http://<server-ip>:3000/api/firmware/<version>",
 "timestamp": 1234567890
}

- ESP32 truy cập URL để tải file OTA
Backend hỗ trợ download binary với header chuẩn:

- Content-Type: application/octet-stream
- Content-Length: ...

4.4. Nguyên lý hoạt động

Hệ thống được vận hành theo một chu trình tự động, khép kín, gồm bốn giai đoạn chính: giám sát – xử lý – kích hoạt – gửi cảnh báo IoT. Các giai đoạn này diễn ra liên tục, đảm bảo khả năng phát hiện sớm và phản ứng kịp thời trước các nguy cơ rò rỉ khí gas hoặc cháy nổ.
Toàn bộ nguyên lý hoạt động được minh hoạ bằng sơ đồ luồng xử lý như sau:



Sơ đồ luồng hoạt động

1. Khởi tạo hệ thống

Khi được cấp nguồn, ESP32 thực hiện các bước khởi tạo bao gồm:

- Khởi động WiFi để kết nối với mạng nội bộ.
- Khởi động MQTT Client, đăng ký (subscribe) các topic điều khiển từ server.
- Khởi chạy các cảm biến MQ2 và cảm biến lửa.

Sau giai đoạn này, thiết bị chuyển sang trạng thái hoạt động liên tục.

2. Thu thập dữ liệu cảm biến (Monitoring/Input)

ESP32 liên tục đọc dữ liệu từ:

- Cảm biến MQ2 → giá trị nồng độ khí gas.
- Cảm biến lửa (Flame) → trạng thái có lửa hoặc không.

Dữ liệu được lấy theo chu kỳ định kỳ (vài trăm ms), bảo đảm tính thời gian thực.

3. Xử lý trung tâm và kiểm tra ngưỡng (Processing)
Giá trị cảm biến sau khi thu thập được chuyển vào khối xử lý chính bên trong ESP32.
Tại đây, vi điều khiển thực hiện phép so sánh với ngưỡng cảnh báo (threshold) mà người dùng đã cấu hình từ Web Dashboard.
Hai trường hợp được phân nhánh:
(A) Trường hợp vượt ngưỡng (Có nguy cơ)
Hệ thống xác định rằng tình trạng đang nguy hiểm — ví dụ:

- Nồng độ khí gas cao bất thường.
- Cảm biến lửa phát hiện tín hiệu ngọn lửa.

Trong trường hợp này, hệ thống chuyển sang chế độ phản ứng khẩn cấp.
(B) Trường hợp không vượt ngưỡng (An toàn)
Nếu môi trường ở trạng thái bình thường, ESP32 vẫn gửi dữ liệu định kỳ lên server để web dashboard hiển thị, đồng thời quay lại vòng lặp giám sát.

4. Kích hoạt hành động tại chỗ (Actuation)
Khi có nguy cơ, ESP32 tự động kích hoạt các thiết bị chấp hành nhằm giảm thiểu rủi ro:

- Bật còi và đèn báo động để cảnh báo trực tiếp.
- Điều khiển servo mở cửa thông gió hoặc đóng van gas.
- Kích hoạt relay để bật quạt hút khí hoặc bật các thiết bị xử lý tại chỗ khác.

Đây là phản ứng thời gian thực và không phụ thuộc Internet, đảm bảo an toàn ngay cả khi mất kết nối mạng.

5. Gửi cảnh báo IoT lên server qua MQTT
Song song với phản ứng tại chỗ, ESP32 gửi thông báo sự cố lên server qua MQTT với các thông tin:

- Mức gas hiện tại.
- Trạng thái lửa.
- Trạng thái relay, servo.
- Thời điểm xảy ra cảnh báo.

MQTT broker tiếp nhận và chuyển dữ liệu đến backend, sau đó WebSocket sẽ cập nhật ngay cho Web Dashboard.

6. Cập nhật thời gian thực lên Web Dashboard
Backend đẩy trạng thái mới của thiết bị lên Web Dashboard giúp người dùng:

- Quan sát được sự cố ngay trên trình duyệt.
- Xem được ngưỡng cảnh báo, thông số MQ2, trạng thái lửa.
- Nhận các thông báo "thiết bị đang cảnh báo" hoặc “vượt ngưỡng an toàn”.
- Ra lệnh điều khiển thủ công (tắt còi, đóng cửa, bật quạt…).

Người dùng có thể theo dõi thiết bị từ xa mà không cần sử dụng phần mềm thứ ba.

7. Lặp lại chu trình
Sau khi xử lý xong, hệ thống quay lại bước ban đầu để tiếp tục:

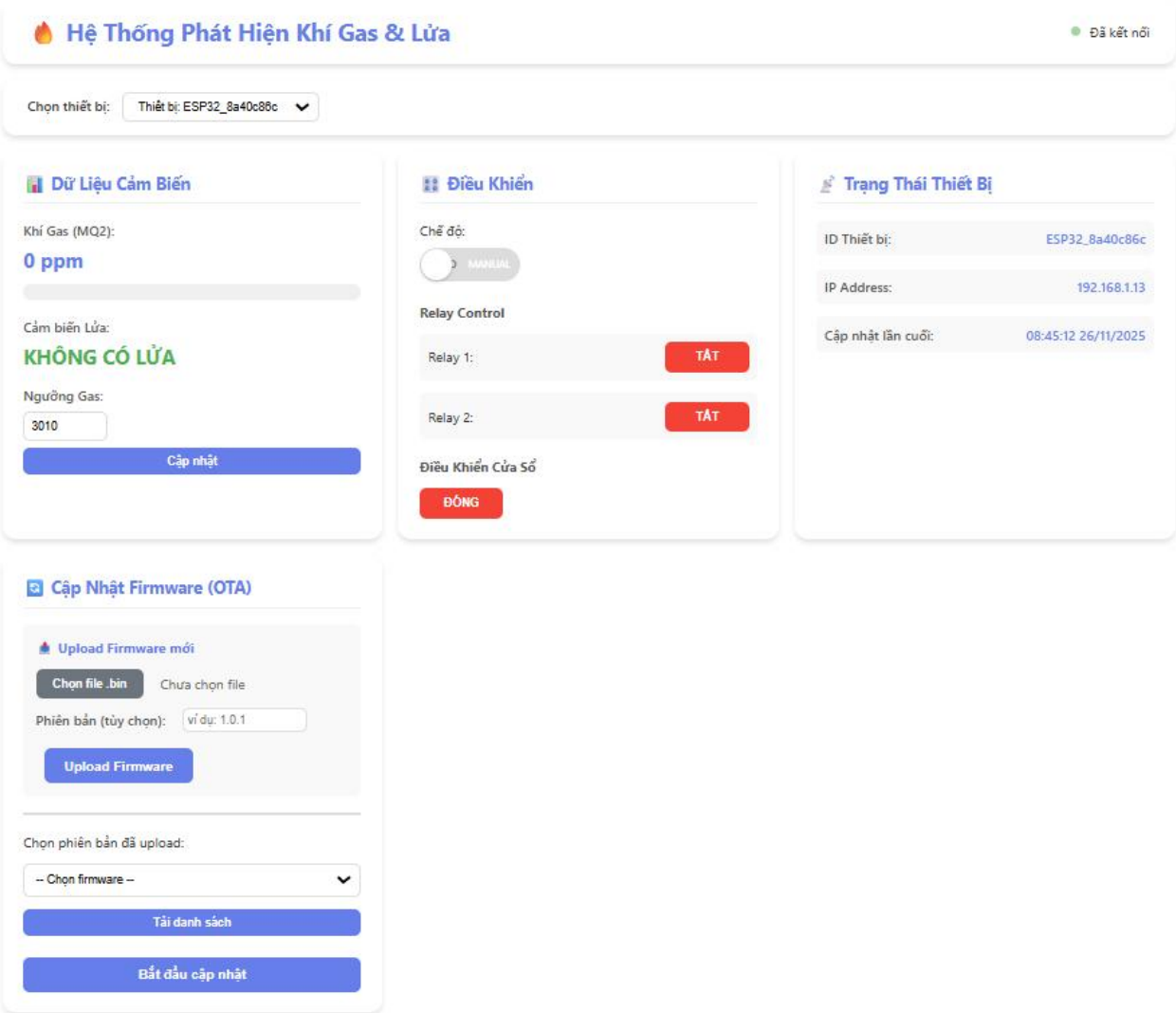
- Thu thập cảm biến
- So sánh ngưỡng
- Xử lý
- Gửi dữ liệu

Vòng lặp này diễn ra liên tục, đảm bảo khả năng giám sát 24/7.

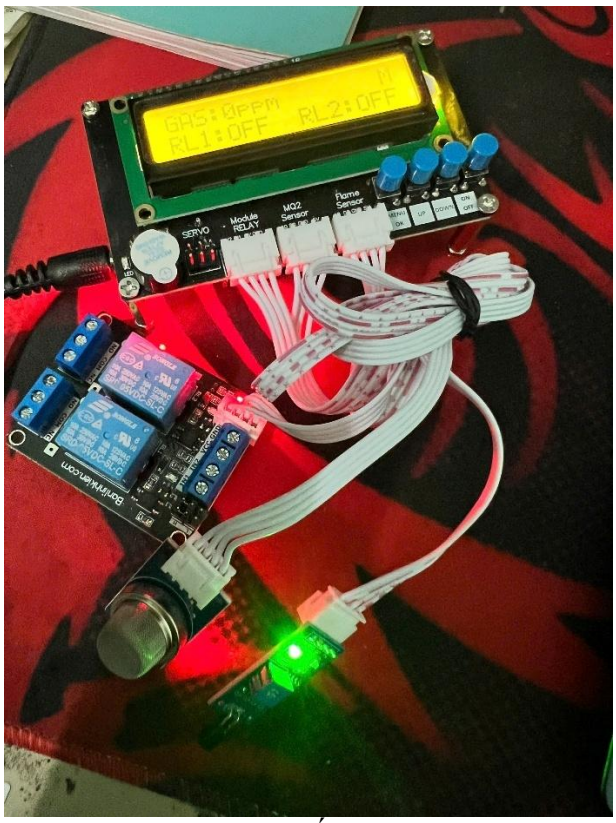
V. KẾT QUẢ ĐẠT ĐƯỢC

5.1. Hoạt động hệ thống

5.1.1. Giao diện web điều khiển



Giao diện web điều khiển

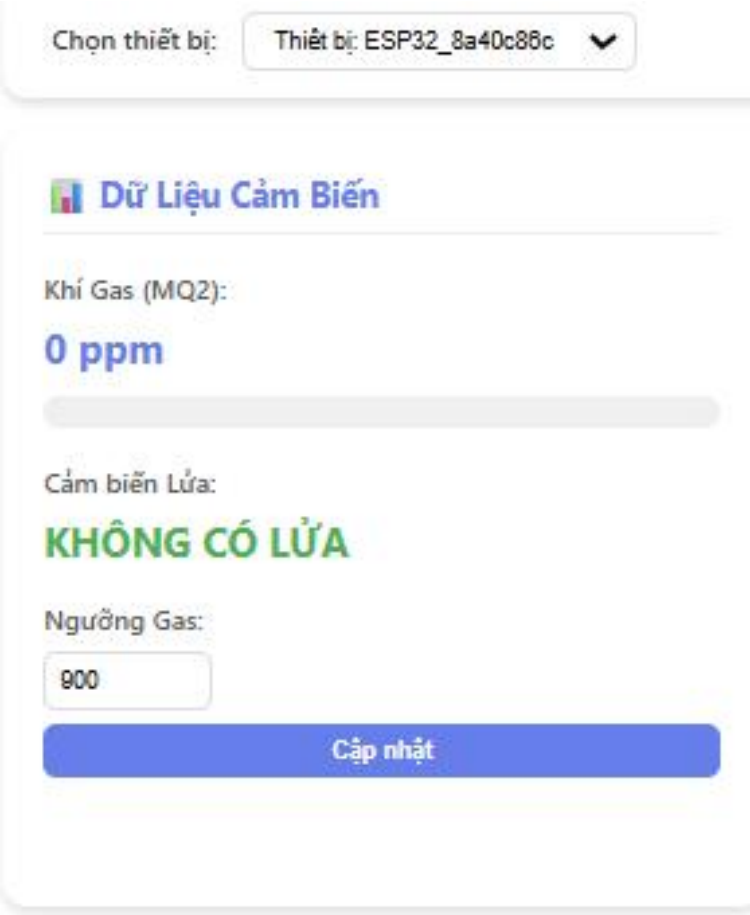


Hình ảnh thiết bị (Demo)

Kết quả giao diện Web điều khiển (Dashboard)

- Hiển thị realtime giá trị khí gas (MQ2) và trạng thái cảm biến lửa.
- Cho phép thay đổi ngưỡng cảnh báo trực tiếp trên giao diện.
- Điều khiển đầy đủ: chế độ Auto/Manual, hai relay và servo cửa sổ.
- Hiển thị thông tin thiết bị: ID, địa chỉ IP, thời gian cập nhật cuối.
- Tích hợp chức năng OTA: upload firmware, chọn phiên bản và gửi lệnh cập nhật.
- Kết nối WebSocket hoạt động ổn định, hiển thị trạng thái “Đã kết nối”.

5.1.2. Giám sát và cập nhật ngưỡng nồng độ khí gas



Cập nhật ngưỡng thành 900

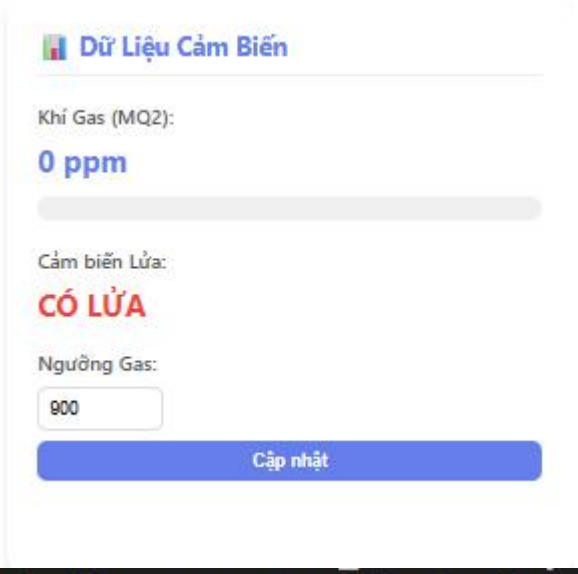
```
[MQTT] Control sent to ESP32_8a40c86c: {"threshold":3010}
[MQTT] Control sent to ESP32_8a40c86c: {"threshold":900}
```

Cập nhật thành công

Kết quả chức năng cập nhật ngưỡng cảnh báo

- Giao diện cho phép người dùng nhập trực tiếp giá trị ngưỡng khí gas mới.
- Ngưỡng cũ (3010 ppm) đã được thay đổi thành 900 ppm và cập nhật thành công lên thiết bị.
- Dashboard hiển thị đúng giá trị ngưỡng mới ngay sau khi cập nhật.
- ESP32 nhận giá trị mới qua WebSocket/MQTT và áp dụng ngay trong quá trình xử lý.
- Việc cập nhật ngưỡng không làm gián đoạn hoạt động giám sát của hệ thống.

5.1.3. Cảnh báo khí gas và phát hiện lửa

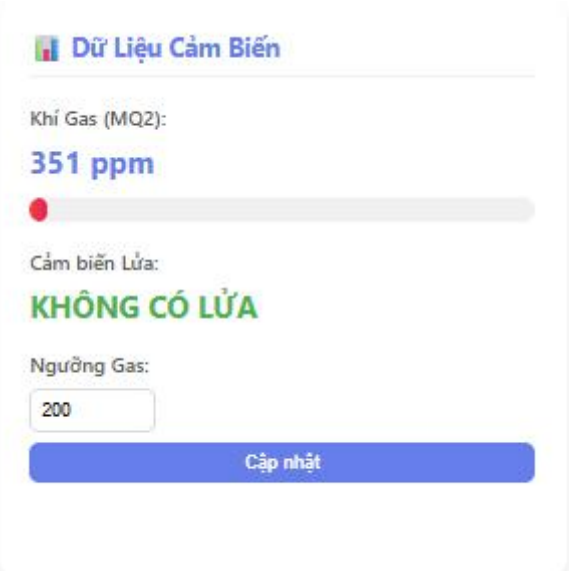



```
[MQTT] device/ESP32_8a40c86c/alert: Fire detected
[MQTT] device/ESP32_8a40c86c/alert: Fire detected
```

Phát hiện có lửa

Kết quả thử nghiệm phát hiện lửa

- Cảm biến lửa đã phát hiện chính xác tín hiệu hồng ngoại và chuyển trạng thái sang "CÓ LỬA".
- Giao diện Dashboard hiển thị cảnh báo rõ ràng bằng chữ đỏ, giúp người dùng dễ dàng nhận biết nguy hiểm.
- Hệ thống lập tức gửi dữ liệu cảnh báo lên server qua MQTT.
- Trạng thái được cập nhật realtime trên Web nhờ WebSocket.
- Thiết bị sẵn sàng kích hoạt các hành động cảnh báo (còi, relay, servo) theo logic xử lý của ESP32.



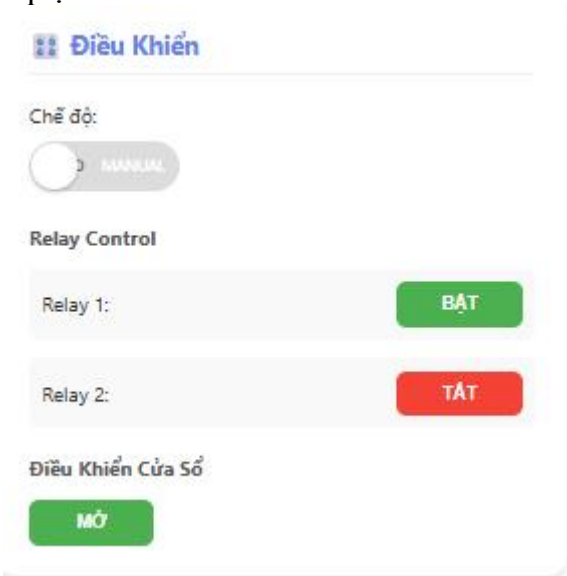

```
[MQTT] device/ESP32_8a40c86c/alert: Gas exceeds permissible limits
```

Cảnh báo khí gas vượt ngưỡng

Kết quả thử nghiệm cảnh báo khí gas vượt ngưỡng

- Giá trị MQ2 tăng lên 351 ppm, vượt ngưỡng cảnh báo đã cài đặt (200 ppm).
- Dashboard hiển thị dữ liệu khí gas theo thời gian thực và đánh dấu mức nguy hiểm bằng thanh cảnh báo màu đỏ.
- Cảm biến lửa vẫn ở trạng thái an toàn (KHÔNG CÓ LỬA).
- Hệ thống lập tức gửi cảnh báo lên server qua MQTT với thông điệp: "Gas exceeds permissible limits".
- Cảnh báo được backend ghi nhận và chuyển tiếp lên Web Dashboard thông qua WebSocket.
- Điều này xác nhận hệ thống xử lý đúng logic so sánh ngưỡng và phản hồi tức thời khi giá trị gas vượt mức an toàn.

5.1.4. Điều khiển servo và quạt/còi




```
[MQTT] Control sent to ESP32_8a40c86c: {"autoManual":1}
[MQTT] Control sent to ESP32_8a40c86c: {"relay1":1,"relay2":false,"autoManual":0}
[MQTT] Control sent to ESP32_8a40c86c: {"relay1":true,"relay2":1,"autoManual":0}
[MQTT] Control sent to ESP32_8a40c86c: {"window":1,"autoManual":0}
[MQTT] Control sent to ESP32_8a40c86c: {"relay1":true,"relay2":0,"autoManual":0}
```

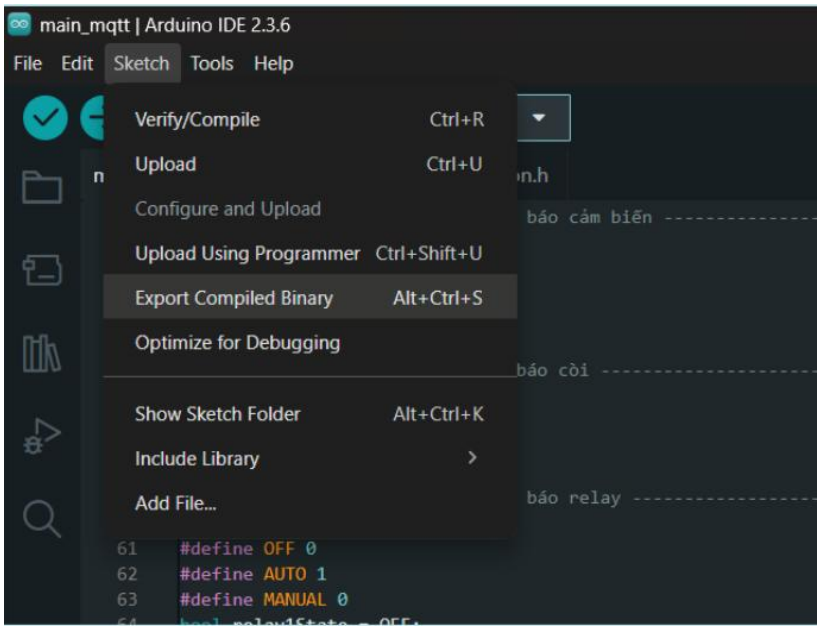
Điều khiển khối chấp hành (quạt, còi, cửa)

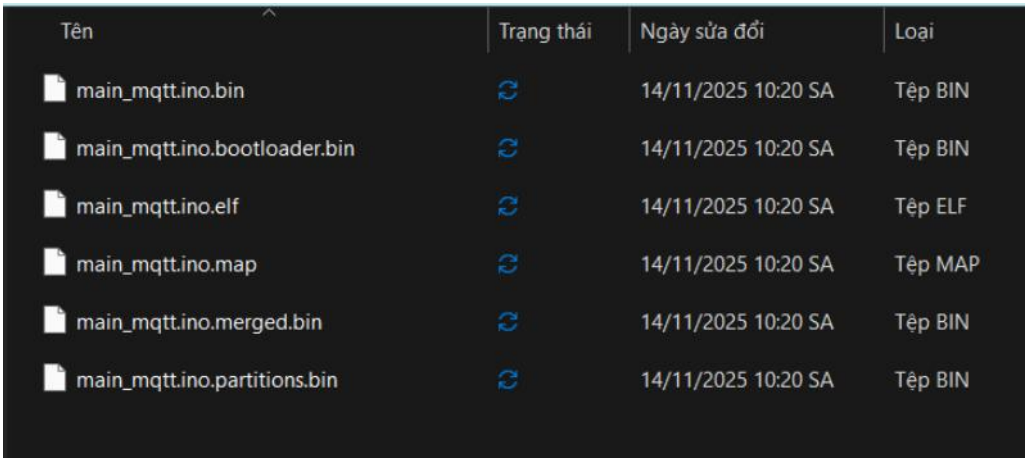
Kết quả điều khiển các thiết bị chấp hành (quạt, còi, cửa/servo)

- Dashboard cho phép chuyển đổi giữa chế độ Auto và Manual để kiểm soát thiết bị linh hoạt.
- Nút điều khiển Relay 1 (quạt) và Relay 2 (còi) phản hồi ngay lập tức khi người dùng nhấn BẬT/TẮT.
- Các lệnh điều khiển được gửi xuống ESP32 qua MQTT và hiển thị trong log server, thể hiện rõ giá trị gửi đi như:
  - {"relay1":1, "relay2":false, "autoManual":0}
  - {"window":1, "autoManual":0}
- Servo cửa được điều khiển chính xác thông qua nút "MỞ/ĐÓNG", và trạng thái cửa được cập nhật realtime.
- Các thiết bị chấp hành phản hồi đúng với giá trị điều khiển, xác nhận hệ thống hoạt động ổn định và không có độ trễ đáng kể.

5.2. Cập nhật firmware từ xa

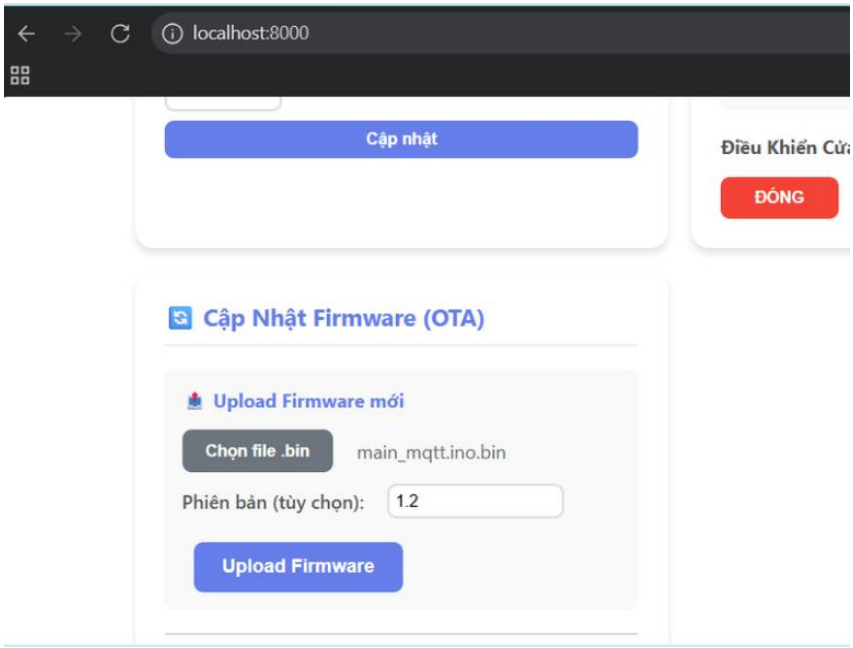
Bước 1: Chuẩn bị bản cập nhật mới trong và xuất từ file C/C++ sang file .bin trong Arduino IDE

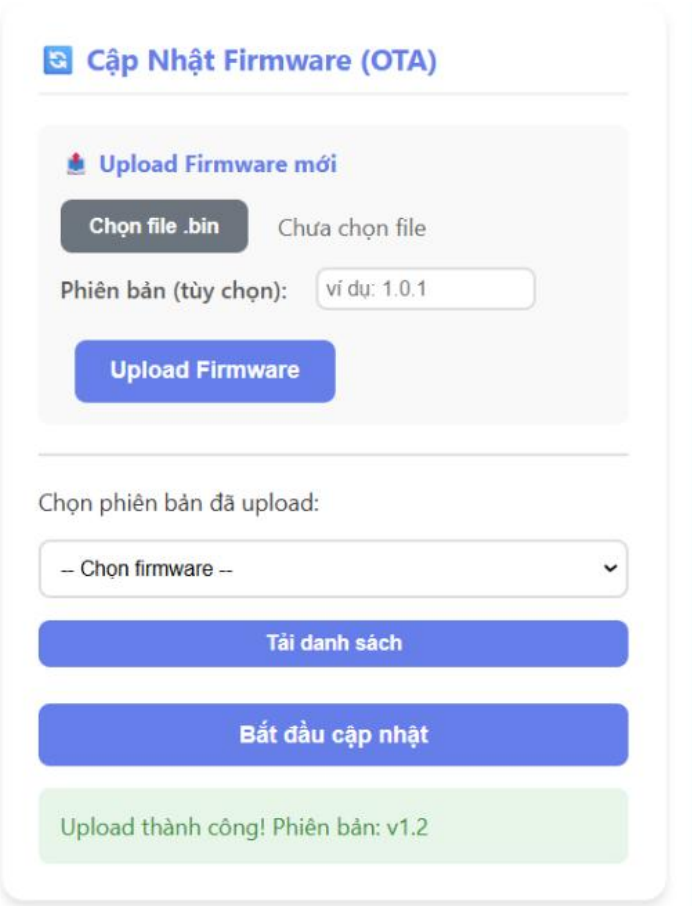




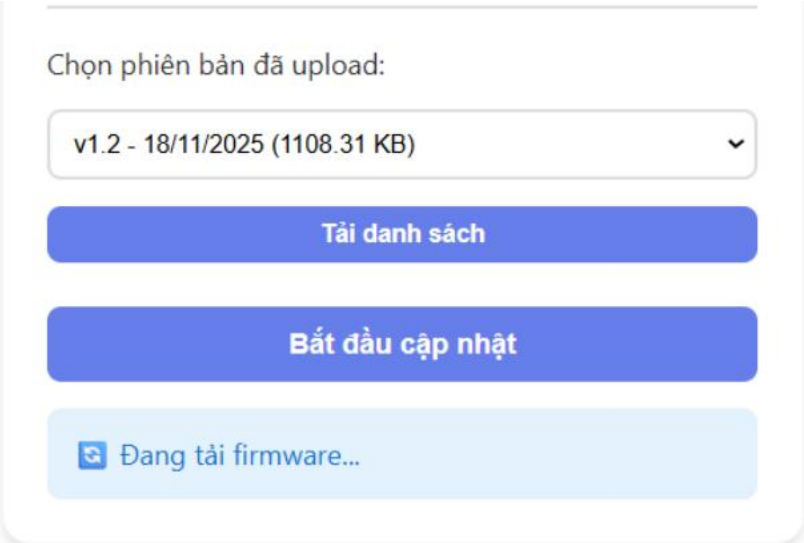
Tên	Trạng thái	Ngày sửa đổi	Loại
main_mqtt.ino.bin		14/11/2025 10:20 SA	Tệp BIN
main_mqtt.ino.bootloader.bin		14/11/2025 10:20 SA	Tệp BIN
main_mqtt.ino.elf		14/11/2025 10:20 SA	Tệp ELF
main_mqtt.ino.map		14/11/2025 10:20 SA	Tệp MAP
main_mqtt.ino.merged.bin		14/11/2025 10:20 SA	Tệp BIN
main_mqtt.ino.partitions.bin		14/11/2025 10:20 SA	Tệp BIN

Bước 2: Trên trang web điều khiển, chọn file .bin vừa tạo và upload. Firmware được lưu vào danh sách các phiên bản để có thể bảo trì về sau





Bước 3: Chọn firmware từ danh sách các phiên bản đã lưu và bắt đầu cập nhật. ESP32 sẽ tự động tải về firmware và cập nhật từ xa mà không cần kết nối dây.




```
[Network] Using interface Wi-Fi: 192.168.1.9
[OTA] Firmware URL: http://192.168.1.9:3000/api/firmware/1.2
[OTA] Device ESP32_8a40c86c: started - OTA update started
[OTA] Serving firmware: C:\Users\hienn\OneDrive\Máy tính\IOT\BTL_IOT_Nhom10\Source Code\backend\firmware\firmware_v17634781125
4.bin (1134912 bytes)
```

VI. KẾT LUẬN – HƯỚNG MỞ RỘNG

6.1. Kết luận

Trong khuôn khổ môn học IoT, nhóm em đã xây dựng thành công một hệ thống giám sát và cảnh báo rò rỉ khí gas – phát hiện lửa dựa trên nền tảng ESP32, giao thức MQTT, và Web Dashboard điều khiển thời gian thực. Hệ thống hoạt động theo một chu trình khép kín: giám sát – xử lý – kích hoạt – gửi cảnh báo, đảm bảo phản ứng nhanh trước các nguy cơ cháy nổ.

Về mặt kỹ thuật, đồ án đã hoàn thiện đầy đủ các thành phần:

- Phần cứng: MQ2, Flame Sensor, Relay, Servo và còi báo được điều khiển ổn định, không nhiễu, phản hồi nhanh.
- Firmware: ESP32 xử lý logic ngưỡng, đọc cảm biến liên tục, gửi dữ liệu và nhận lệnh chính xác qua MQTT.
- Backend: Hệ thống Node.js tích hợp MQTT Broker và WebSocket, vận hành mượt, hỗ trợ nhiều thiết bị cùng lúc.
- Frontend: Web Dashboard hiển thị realtime, điều khiển từ xa, hỗ trợ tính năng OTA Firmware Update.

6.2. Hướng mở rộng

Dựa trên hệ thống hiện tại, nhóm có thể tiếp tục phát triển theo các hướng sau:

1. Tích hợp thêm cảm biến môi trường

Có thể bổ sung cảm biến CO, CO_2, độ ẩm, nhiệt độ, chất hữu cơ bay hơi (VOC) hoặc cảm biến khói chuyên dụng để mở rộng phạm vi giám sát môi trường.

2. Nâng cấp cơ chế cảnh báo từ xa

Thêm các kênh cảnh báo đa dạng như:

- Gửi SMS, email, Telegram…
- Push Notification qua Firebase Cloud Messaging (FCM).
  Điều này giúp người dùng nhận được thông báo ngay cả khi không mở Web Dashboard.

3. Mở rộng thành hệ thống đa thiết bị (multi-node)

Backend hiện tại đã hỗ trợ nhiều device ID, nên có thể mở rộng thành mạng IoT bao phủ nhiều phòng, tầng hoặc khu vực.

Web Dashboard có thể nâng cấp để hiển thị bản đồ thiết bị hoặc phân nhóm theo từng khu vực.

4. Tăng cường bảo mật giao tiếp

- Thêm mã hóa TLS cho MQTT (MQTTS).
- Xác thực API bằng token hoặc JWT.
- Thêm đăng nhập người dùng vào Web Dashboard.

5. Nâng cấp giao diện Web

- Thiết kế đồ thị lịch sử dữ liệu (gas, lửa).
- Thêm trang thống kê và cảnh báo nâng cao.
- Hỗ trợ dark/light mode.

6. Tối ưu OTA và quản lý phiên bản

- Thêm kiểm tra version giữa thiết bị và server.
- Thêm chức năng rollback firmware khi OTA lỗi.
- Lưu lịch sử cập nhật theo từng thiết bị.

7. Hướng tới triển khai thực tế

Mô hình có thể tích hợp vào nhà bếp, nhà kho, phòng thí nghiệm, hoặc hệ thống công nghiệp với việc thêm:

- Vỏ bảo vệ đạt chuẩn IP.
- Bộ nguồn ổn định và pin dự phòng.
- Van gas điện tử thực tế thay vì servo mô phỏng.